

Chapter 10: Deep Feedforward Networks & Backpropagation

One concept, one exercise per slide — practice as you go

Concept: The Multidimensional Chain Rule

- A deep network is a chain of matrix-valued functions, not scalar ones
- The same chain rule from Chapter 4 applies — just with matrices and vectors instead of scalars
- Backpropagation is not a new algorithm — it is the chain rule applied systematically, layer by layer



Exercise: Why is it accurate to say 'backpropagation is not a new optimization algorithm' when it clearly makes deep learning possible?

Concept: Backward Pass for a Linear Layer

- Forward: $Z = X \cdot W + b$
- $dW = X^T \cdot dZ$
- $db = \text{sum of } dZ \text{ across all rows (the batch dimension)}$
- $dX = dZ \cdot W^T$ (needed to keep propagating backward into earlier layers)



Exercise: A linear layer has X shape (32, 10), W shape (10, 5). If dZ has shape (32, 5), what are the shapes of dW , db , and dX ? Do they match W , b , and X respectively?

Concept: Backward Pass for ReLU and Sigmoid

- ReLU backward: $\text{grad_output} * (\text{input} > 0)$ — pass gradient through only where the input was positive
- Sigmoid backward: $\text{grad_output} * \text{output} * (1 - \text{output})$ — uses the CACHED forward output, not the input
- Every activation's backward rule needs something cached from its forward pass



Exercise: A ReLU layer's cached input was $[-2, 3, 0, 5]$. The incoming gradient is $[1, 1, 1, 1]$. What is the outgoing gradient?

Concept: Softmax + Cross-Entropy — The Elegant Simplification

- Softmax converts logits to probabilities; cross-entropy measures how wrong those probabilities are
- Computed separately, their combined backward pass would be a messy Jacobian product
- Computed TOGETHER, the gradient simplifies beautifully to: $d\text{Logits} = (1/B)(P - Y)$
- P = predicted probabilities, Y = one-hot true labels, B = batch size



Exercise: For one example with true class 1 (of 3 classes) and predicted probabilities $P=[0.2, 0.5, 0.3]$, what is the gradient with respect to the logits (ignore the $1/B$ batch averaging for this single example)?

Concept: Gradient Flow Pathologies

- Vanishing gradients: products of many small Jacobians shrink toward zero across depth
- Exploding gradients: products of many large Jacobians grow without bound across depth
- Both make deep networks difficult or impossible to train without countermeasures
- Initialization, activation choice, normalization, and residual connections are all responses to this



Exercise: A 20-layer sigmoid network trains fine for the first few layers near the output but the earliest layers' weights barely change at all during training. Which gradient pathology is this, and why does it hit early layers hardest?

Concept: Xavier Initialization

- Goal: keep the variance of activations roughly CONSTANT as signal passes through each layer
- Derived from $\text{Var}(z) = n_{\text{in}} \cdot \text{Var}(x) \cdot \text{Var}(w)$ — solving for the weight variance that keeps $\text{Var}(z) \approx \text{Var}(x)$
- Result: $\text{Var}(W) = 2 / (n_{\text{in}} + n_{\text{out}})$ — designed for symmetric activations like tanh



Exercise: Why does Xavier initialization's variance formula include BOTH n_{in} and n_{out} , rather than just n_{in} ?

Concept: He Initialization

- Designed specifically for ReLU, which zeros out roughly half its inputs
- Because ReLU discards half the signal, the remaining variance must be compensated for
- Result: $\text{Var}(W) = 2 / n_{\text{in}}$ — exactly double Xavier's forward-only term



Exercise: Explain, in one sentence, why He initialization uses exactly $2/n_{\text{in}}$ rather than Xavier's more complex $2/(n_{\text{in}}+n_{\text{out}})$ formula.

Concept: The Module Contract

- Every layer implements the same three methods: `forward()`, `backward()`, `parameters()`
- `forward()` computes outputs AND caches whatever `backward()` will need
- `Sequential.backward()` walks layers in REVERSE order — the chain rule runs output-to-input
- This uniform contract is what lets layers compose into arbitrarily deep networks



Exercise: A Sequential model has layers [Linear, ReLU, Linear]. In what order does `forward()` call them, and in what order does `backward()` call them?

Concept: Numerical Gradient Checking

- Central difference: $f'(\theta) \approx [f(\theta+h) - f(\theta-h)] / (2h)$ — more accurate than the one-sided version
- Compare this numerical estimate against your analytical (backprop) gradient
- Use RELATIVE error, not absolute — absolute error is misleading when gradients are tiny
- A correct float64 implementation should show relative error below about $1e-7$



Exercise: Your gradient check reports analytical=0.0031, numerical=0.0029. Is a relative-error tolerance of $1e-7$ likely to be satisfied? What does this suggest about your backward pass?

Concept: Common Bug — Missing Batch Averaging

- If the FORWARD loss averages over the batch (divides by B), the BACKWARD gradient must too
- Forgetting $\text{grad} \neq \text{batch_size}$ makes the effective learning rate scale with batch size
- This is a silent bug — training still runs, just with a batch-size-dependent effective step size



Exercise: You forget to divide the cross-entropy gradient by `batch_size`. If you then double your batch size, what happens to the effective step size of each parameter update, all else equal?

Concept: Common Bug — Mutating Cached Inputs

- Backward methods depend on values CACHED during the forward pass
- If a cached array is mutated in place before backward() runs, the gradient computation reads corrupted data
- Treat every cached tensor as read-only until backward() has consumed it



Exercise: A custom training loop applies an in-place data augmentation to the input array AFTER the forward pass but BEFORE calling backward(). Why is this dangerous if the layer cached a reference to that same array?

Concept: Diagnosing a Training Run

- Activation histograms: watch for ReLUs stuck at all-zero, or sigmoids saturated at 0/1
- Gradient norms per layer: watch for early layers with vastly smaller norms than later layers
- Loss curves: flat-and-high (underfitting), diverging train/val gap (overfitting), NaN (instability)
- None of these prove correctness — but they narrow down WHERE to look for a bug



Exercise: A network's loss suddenly becomes NaN partway through training. Name two of the most likely causes to check first.

Mastery Check: Are You Ready to Move On?

If you can answer every question below confidently, you have mastered Chapter 10.

1. Explain why backpropagation is the chain rule applied systematically, not a separate algorithm.
2. Derive dW , db , and dX for a linear layer $Z = XW + b$ and verify their shapes match W , b , and X .
3. Derive the backward rule for ReLU and for sigmoid, and explain what each one caches from its forward pass.
4. Derive why the combined softmax + cross-entropy gradient simplifies to $(P - Y) / B$.
5. Explain vanishing and exploding gradients and why they hit early layers of a deep network hardest.
6. Derive Xavier initialization's variance formula and explain why it involves both n_{in} and n_{out} .
7. Derive He initialization's variance formula and explain why it differs from Xavier by a factor of 2.
8. Explain the Module contract (forward/backward/parameters) and why Sequential's backward runs in reverse order.
9. Explain why numerical gradient checking uses relative error and central differences, and state a typical tolerance.
10. Diagnose the training symptom caused by forgetting to divide the loss gradient by batch size.
11. Diagnose a bug caused by mutating a cached forward-pass value before backward() runs.
12. Implement a complete Linear + ReLU + Softmax-CrossEntropy MLP from scratch and verify it with a numerical gradient check.
13. Train the scratch MLP on the two-spiral dataset and confirm a linear model could not have solved it.